



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the complete, fixed set of all valid (x, y) positions on the grid and the legal transitions between them (via Up/Down/Left/Right, excluding walls). It exists independently of any search, it's just the map. The search tree is what the algorithm actually builds while exploring: a tree of nodes representing the paths it has tried, rooted at the start state. The key difference is that a state can appear only once in the state space, but without care, the same state could appear as many different nodes in the search tree (reached via different paths), which is exactly what the reached set is there to prevent.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the problem of re-expanding already-visited states. Without it, the search would treat every new path back to a previously-seen cell as a fresh node to expand, converting a finite state space into a potentially infinite search tree (since you can walk back and forth between two open cells indefinitely). This is what turns a Tree Search into a Graph Search. For the DFS agent specifically, without reached, the algorithm could get stuck oscillating between a small loop of cells forever, pushing the same states onto the stack repeatedly and never terminating or reaching the goal.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS uses a FIFO queue, so it expands nodes in strict order of depth, it fully explores all paths of length 1 before any path of length 2, and so on. Since every move in this grid has an identical cost of 1, depth is equivalent to path cost, so the first time BFS reaches the goal, it's guaranteed to be via the shortest possible path. DFS uses a LIFO stack, so it commits fully to one direction and only backtracks after hitting a dead end (a wall or the grid boundary). This means DFS can find a long, winding path to the goal well before a much shorter path is even considered, because it never compares path lengths, it just takes whatever branch it's currently on to its conclusion.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

Time complexity for both is $O(b^d)$, where b is the branching factor (here, up to 4) and d is the depth of the shallowest solution, this is the same for BFS and DFS in the worst case. The real difference is space. BFS must store its entire frontier in memory at once, at depth d , that frontier can hold on the order of b^d nodes, since it explores level-by-level and keeps every node at the current depth before moving to the next. On a 1000x1000 grid, this means BFS's memory usage can explode toward the size of the whole state space before it finds distant food, risking a crash. DFS, by contrast, only needs to store the current path from root to leaf plus the unexplored siblings along that path, its space complexity is $O(b \cdot d)$, linear in depth rather than exponential. So DFS is far more memory-efficient at scale, even though the path it finds is typically much worse in quality (not optimal, and often significantly longer).

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING